

AI-Based Cyber Attack Detection in the Internet of Medical Things (IoMT)

Complete Technical Build Document

Group No: 22 | **Class:** BE Computer, Division B **Institute:** Sandip Institute of Technology and Research Centre, Nashik (Autonomous, affiliated to SPPU Pune)
Academic Year: 2026–27 **Document version:** 1.0 — Build Reference Document **Purpose:** This is the single reference document for building the project. It covers rationale, solution design, system architecture, technology stack, dataset strategy, all ML models with model-wise functionality, the complete database schema, module-wise functionality, APIs, testing, evaluation, and the project plan.

0. How to read this document

Section	What it gives you
1-3	Why the project exists, what is broken today, what we are building
4-5	System architecture and the exact technology stack
6-7	Datasets and feature engineering
8-9	Every ML model, what it does, and how they combine
10	Complete database design with all tables and DDL
11-14	Module-by-module functionality, API contracts, dashboard, alerting
15-18	Testbed, deployment, testing, evaluation metrics
19-21	Timeline, team allocation, risks
22-24	Improvements over the synopsis, corrected references, viva preparation

1. Why this project was taken

1.1 The domain problem

Hospitals have quietly become one of the most connected — and most fragile — computing environments in existence. A single ICU bed today can involve an infusion pump, a ventilator, a multi-parameter patient monitor, a pulse oximeter, a smart bed, and a bedside gateway, all speaking over Wi-Fi, MQTT, Bluetooth/BLE, or HL7 over TCP. Collectively these are the **Internet of Medical Things (IoMT)**.

Four properties make IoMT uniquely hard to secure:

- Devices cannot be patched freely.** A ventilator's firmware is regulatory-certified. You cannot push a security update the way you patch a laptop. Vulnerabilities therefore stay open for years.
- Availability outranks confidentiality.** In IT, you isolate a compromised machine. In a hospital, disconnecting a monitor attached to a patient can be lethal. Classic incident response does not apply.
- Devices are computationally weak.** Most IoMT endpoints cannot run endpoint agents, encryption at full strength, or local security software. Security must live in the network, not on the device.
- The attack surface is physiological, not just digital.** An attacker who alters a transmitted SpO₂ value from 88% to 98% does not steal anything — but a clinician may withhold oxygen. The harm is medical, not financial.

1.2 Why the existing approach fails

Existing approach	Why it is insufficient for IoMT
Signature-based IDS (Snort, Suricata rules)	Detects only known attack signatures. Zero-day and mutated attacks pass through. Requires constant rule updates.
Firewall / ACL only	Blocks by IP and port. Cannot see that a legitimate MQTT connection is being used for a flood or an injection.
Manual SOC monitoring	An 80-bed hospital generates millions of flow records per day. Human monitoring does not scale.
Generic IT antivirus	Cannot be installed on embedded medical devices at all.
Existing IoT ML research	Trained on generic IoT datasets (smart home, industrial). Does not model medical protocols, device profiles, or physiological plausibility.

The gap: there is no lightweight, explainable, real-time detection layer that understands *both* IoMT network behaviour *and* the medical plausibility of the data being carried.

1.3 Why this is a good final-year project

- It combines two high-demand domains — Artificial Intelligence and Cybersecurity — in a healthcare context.
- It is fully buildable with free, public, peer-reviewed datasets and open-source tooling, on ordinary laptops.
- It produces a demonstrable working artefact (live dashboard, live attack simulation, live detection) rather than only a report.
- It maps cleanly to **SDG 3** (health), **SDG 9** (resilient infrastructure), and **SDG 16** (strong, trustworthy institutions).
- It is publishable: the multi-stage + explainability + physiological-plausibility combination is a genuine, defensible contribution.

2. Problem statement (build-ready version)

Design, implement and evaluate a **hybrid, multi-stage, explainable AI-based Intrusion Detection System (IDS)** for IoMT networks that ingests live network flow telemetry and device physiological telemetry, detects whether traffic is benign or malicious in real time, classifies the specific attack family when malicious, flags previously unseen (zero-day) behaviour through unsupervised anomaly modelling, verifies that reported patient vitals are physiologically plausible, explains every decision to a human operator, and raises prioritised, non-disruptive alerts — while keeping per-flow inference latency low enough for practical deployment at a hospital network edge.

2.1 Scope boundaries (state these clearly in the report)

In scope - Detection and classification of network-borne attacks against IoT devices - Zero-day / anomaly detection - Physiological plausibility checking of vitals - Explainability, alerting, dashboard, reporting

Out of scope - Automatic blocking or shutting down of medical devices (we alert and recommend; we do not cut off a live device) - Deep packet payload decryption - Firmware-level or hardware-level attacks - Real clinical patient data (we use public research datasets and a synthetic testbed only)

3. Proposed solution

3.1 Solution in one paragraph

We build a **network-edge IDS appliance** (software, runs on a laptop / Raspberry Pi / VM sitting beside the hospital gateway). It passively captures traffic to and from IoT devices, converts packets into statistical flow records, and pushes each flow through a **five-stage detection pipeline**. Stage 1 answers "is this benign or malicious?" fast. Stage 2 answers "which attack family is this?" Stage 3 answers "even if the classifier says benign, is this behaviour abnormal for this specific device?" Stage 4 answers "do the vitals inside this traffic make medical sense?" Stage 5 answers "why did the system decide this?" Results are written to a database, streamed to a React dashboard over WebSocket, and turned into severity-ranked alerts with recommended (never automatic) mitigation actions.

3.2 The five detection stages



3.3 What makes this better than a plain "train a classifier on a dataset" project

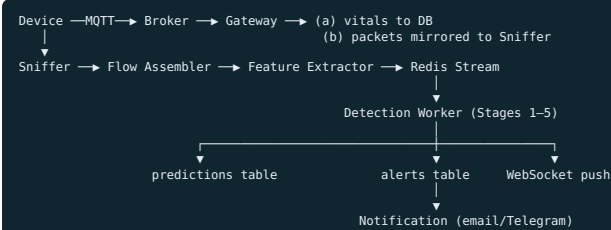
Ordinary project	This project
One model, one accuracy number	Five cooperating stages, each with a defined job
Detects only trained attack classes	Autoencoder stage catches unseen behaviour
Black-box output	SHAP explains every single alert
Network-only view	Adds physiological plausibility — unique to IoT
Offline CSV evaluation	Live capture → live inference → live dashboard
No operational thinking	Severity scoring, recommended actions, safety rule of never auto-disconnecting a device

4. System architecture

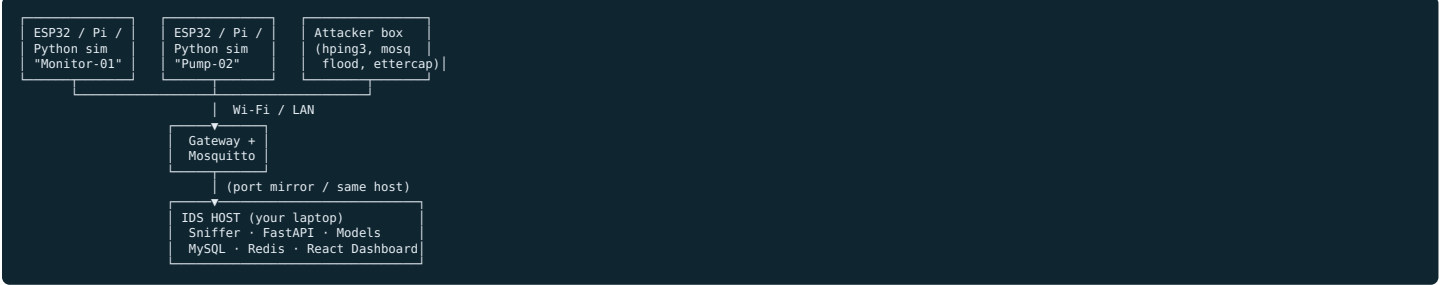
4.1 Layered architecture

- Layer 0 — IoT Device Layer** Real or simulated medical devices: patient monitor, infusion pump, pulse oximeter, ECG sensor, smart thermometer, ventilator simulator. Protocols: MQTT over TCP, HTTP/REST, BLE.
- Layer 1 — Edge Gateway Layer** Mosquitto MQTT broker + a Python gateway service. Devices publish to topics like `hospital/ward1/bed3/vitals`. The gateway normalises payloads and forwards them to the backend. This is also the point where an attacker in our testbed sits (MITM).
- Layer 2 — Capture & Feature Extraction Layer** Scapy / tshark sniffs the interface in promiscuous mode. A flow assembler groups packets into bidirectional flows keyed by `(src_ip, dst_ip, src_port, dst_port, protocol)` with a flow timeout, and computes statistical features (see §7).
- Layer 3 — Detection Layer (the AI core)** FastAPI service holding all loaded models in memory. Exposes `/api/v1/detect` and consumes from a Redis stream for batch throughput. This is where Stages 1–5 execute.
- Layer 4 — Persistence Layer** MySQL 8 for relational and operational data (devices, alerts, users, model registry, audit). Redis for the hot streaming buffer, rate limiting, and pub/sub. Optional: file storage for `.pcap` evidence and serialised models.
- Layer 5 — Application/API Layer** FastAPI REST + WebSocket. JWT authentication, role-based access control, alert lifecycle management, reporting endpoints.
- Layer 6 — Presentation Layer** React (Vite) + Tailwind dashboard: live traffic view, alert console, device inventory, model performance page, SHAP explanation panel, reports.

4.2 Data flow (end to end)



4.3 Deployment topology (demo setup)



5. Technology stack

5.1 Full stack table

Layer	Technology	Version	Why this choice
Language (AI/backend)	Python	3.11	Best ML ecosystem; 3.11 is fast and stable with TF/Torch
Backend framework	FastAPI	0.110+	Async, native WebSocket, auto OpenAPI docs, very fast
ASGI server	Uvicorn (+ Gunicorn workers)	latest	Production-grade async serving
Classical ML	scikit-learn	1.4+	RF, SVM, Isolation Forest, preprocessing, metrics
Gradient boosting	XGBoost	2.0+	Best accuracy/latency ratio on tabular flow data
Gradient boosting (alt)	LightGBM	4.x	Faster training on large flow sets; used for comparison
Deep learning	TensorFlow + Keras	2.15+	Autoencoder, CNN-BiLSTM, LSTM forecaster
Explainability	SHAP	0.45+	TreeSHAP is exact and fast for tree models
Explainability (local)	LIME	0.2+	Secondary per-instance explanation for comparison
Imbalance handling	imbalanced-learn (SMOTE, ADASYN)	0.12+	Attack classes are heavily minority
Data handling	Pandas, NumPy	2.x / 1.26+	Preprocessing, feature engineering
Visualisation (analysis)	Matplotlib, Seaborn	latest	Confusion matrices, ROC, feature importance plots
Packet capture	Scapy	2.5+	Pure-Python sniffing and crafting; easy to script
Packet capture (CLI)	tshark / Wireshark	4.x	Verification and pcap inspection
Flow feature extraction	CICFlowMeter (or custom Python extractor)	—	Produces the same feature space as CIC datasets
MQTT broker	Eclipse Mosquitto	2.0+	Standard IoMT/IoT protocol broker, lightweight
MQTT client	paho-mqtt	2.x	Device simulators and gateway
Primary database	MySQL	8.0	Relational integrity, familiar, good tooling
Cache / stream buffer	Redis	7.x	Redis Streams for flow queue, pub/sub for live alerts
ORM / migrations	SQLAlchemy 2.0 + Alembic	latest	Typed models, versioned schema migrations
Auth	JWT (python-jose) + passlib/bcrypt	latest	Stateless auth, hashed credentials
Frontend	React 18 + Vite	latest	Fast dev loop, component model
UI styling	Tailwind CSS 3	latest	Rapid, consistent UI
Charts	Recharts / Chart.js	latest	Traffic timelines, attack distribution, metric charts
Realtime UI	native WebSocket API	—	Live alert push without polling
Attack simulation	hping3, Nmap, Ettercap/Bettercap, MQTTSA, custom Scapy scripts	—	Generates DoS/DDoS, recon, MITM, MQTT floods safely in the lab
Containerisation	Docker + Docker Compose	latest	One-command reproducible demo
Version control	Git + GitHub	—	Collaboration, branch per module
Experiment tracking	MLflow (optional)	2.x	Compare model runs, store artefacts, register versions
Notebooks	Jupyter Lab	latest	Model experimentation and EDA
IDE	VS Code / PyCharm	latest	Development
Documentation	Markdown + LaTeX/Word	—	Report and paper
Testing	pytest, pytest-asyncio	latest	Unit and integration tests
API testing	Postman / Thunder Client	—	Endpoint verification

5.2 Hardware requirements

Item	Minimum	Recommended
CPU	Intel i5 (4-core) / Ryzen 5	i7 / Ryzen 7, 8-core
RAM	8 GB	16 GB (32 GB if training deep models on full CICIoMT2024)
Storage	256 GB SSD	512 GB SSD (raw pcaps are large)
GPU	not mandatory	NVIDIA GTX 1650 / RTX 3050 4 GB+ or free Google Colab / Kaggle GPU
Devices (optional but impressive)	—	2× ESP32 or 1× Raspberry Pi 4 + MAX30100 (SpO ₂ /HR), DS18B20 (temp), AD8232 (ECG)
Network	Wi-Fi router / managed switch with port mirroring	same

Practical note: if you have no GPU, train the deep models (autoencoder, CNN-BiLSTM, LSTM) on **Google Colab free tier**, download the `.h5` / `.keras` files, and run inference locally on CPU. Inference is cheap; only training needs the GPU.

5.3 Project folder structure

```
iomt-ids/
├── docker-compose.yml
├── README.md
├── requirements.txt
├── .env.example
├── data/
│   ├── raw/           # downloaded datasets, pcaps
│   ├── interim/       # merged, cleaned
│   ├── processed/     # train/val/test splits
│   └── scalars/       # fitted StandardScaler, LabelEncoder .pkl
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_preprocessing.ipynb
│   ├── 03_binary_models.ipynb
│   ├── 04_multiclass_models.ipynb
│   ├── 05_autoencoder.ipynb
│   ├── 06_vitals_lstm.ipynb
│   └── 07_shap_analysis.ipynb
├── ml/
│   ├── preprocessing.py
│   ├── feature_engineering.py
│   ├── train_binary.py
│   ├── train_multiclass.py
│   ├── train_autoencoder.py
│   ├── train_vitals_lstm.py
│   ├── evaluate.py
│   ├── explain.py
│   └── registry.py      # load/save models, version metadata
├── models/             # serialised artefacts
│   ├── binary_rf_v1.pkl
│   ├── multiclass_xgb_v1.pkl
│   ├── autoencoder_v1.keras
│   ├── vitals_lstm_v1.keras
│   ├── scaler_v1.pkl
│   └── label_encoder_v1.pkl
├── capture/
│   ├── sniffer.py
│   ├── flow_assembler.py
│   └── feature_extractor.py
├── simulator/
│   ├── device_monitor.py # publishes normal vitals
│   ├── device_pump.py
│   ├── attacker_dos.py
│   ├── attacker_mqtt_flood.py
│   ├── attacker_mitm_inject.py
│   └── attacker_recon.py
├── backend/
│   ├── main.py
│   ├── config.py
│   ├── database.py
│   ├── models/        # SQLAlchemy ORM classes
│   ├── schemas/       # Pydantic request/response
│   ├── routers/
│   │   ├── auth.py
│   │   ├── devices.py
│   │   ├── flows.py
│   │   ├── detections.py
│   │   ├── alerts.py
│   │   ├── models.py
│   │   ├── reports.py
│   │   └── ws.py
│   ├── services/
│   │   ├── detection_service.py
│   │   ├── alert_service.py
│   │   ├── explain_service.py
│   │   └── notify_service.py
│   └── workers/
│       └── detection_worker.py
├── frontend/
│   ├── src/
│   │   ├── pages/      # Dashboard, Alerts, Devices, Models, Reports
│   │   ├── components/
│   │   ├── hooks/useWebSocket.js
│   │   ├── api/client.js
│   │   └── vite.config.js
├── migrations/         # Alembic
├── tests/
└── docs/
    ├── architecture.md
    ├── er_diagram.png
    └── api_spec.md
```

6. Dataset strategy

Your synopsis listed CICIDS2017, CICIoT2023, NSL-KDD, UNSW-NB15 and BoT-IoT. Those are all **generic IT/IoT** datasets. For an *IoMT* project you must lead with IoMT-specific datasets, otherwise the examiner's first question will be "where is the medical part?"

6.1 Recommended dataset set

#	Dataset	Role in this project	Key facts	Source
D 1	CICIoMT2024 (Canadian Institute for Cybersecurity, UNB)	Primary — main training/testing corpus	18 attacks against a testbed of 40 IoMT devices (25 real + 15 simulated) across Wi-Fi, MQTT and Bluetooth; attacks grouped into 5 families: DDoS, DoS, Recon, MQTT, Spoofing. ~3.2M labelled instances, 46 features. Supports 2-class, 6-class and 19-class labelling. Also includes device <i>profiling</i> captures.	unb.ca/cic/datasets/iomt-dataset-2024.html
D 2	WUSTL-EHMS-2020 (Washington University in St. Louis)	Secondary — enables the physiological-plausibility stage	Built on the Enhanced Healthcare Monitoring System testbed. 16,318 records (14,272 normal / 2,046 attack), 44 features = 35 network-flow + 8 patient-biometric + 1 label. Attacks are man-in-the-middle: spoofing and data injection/alteration.	cse.wustl.edu/~jain/ehms/
D 3	CICIoT2023	Transfer/augmentation + generalisation test	34 classes (1 benign + 33 attacks) reducible to 8 categories: DDoS, DoS, Recon, Brute Force, Spoofing, Web, Mirai, Benign. 46 features — feature space aligns well with CICIoMT2024.	unb.ca/cic/datasets/iotdataset-2023.html
D 4	ECU-IoHT	Small IoHT-specific cross-check	Healthcare IoT attack traces (ARP spoof, DoS, Nmap scan, Smurf)	Edith Cowan University
D 5	Your own testbed capture	Live demo + novelty	Captured from your Mosquito/ESP32 setup with your own attack scripts	generated by you

6.2 Why this combination is defensible

- **D1** gives you scale, protocol diversity, and a modern IoMT-native benchmark — this is what you report headline accuracy on.
- **D2** is the only dataset that pairs **network features with patient biometrics**, which is exactly what Stage 4 (physiological plausibility) needs. Without it, the "medical" claim in your title is decorative.
- **D3** lets you demonstrate **cross-dataset generalisation** — train on D1, test on D3 — which is the single most impressive result you can show at BE level, because most published work never does it.
- **D5** proves the system actually runs live, which is what the examiner will judge in the demo.

6.3 Attack taxonomy used in this project

Family	Concrete attacks	Detection signal
DDoS	ICMP flood, UDP flood, TCP SYN flood, MQTT publish flood (distributed)	Extremely high packet rate, many sources, low flow duration, uniform packet sizes
DoS	Single-source SYN/UDP/ICMP flood, Slowloris	High rate from one source, high SYN ratio, half-open connections
Recon	Nmap port scan, ping sweep, OS fingerprinting, MQTT topic enumeration	Many short flows, many distinct destination ports, low bytes per flow, high RST ratio
MQTT-specific	CONNECT flood, malformed publish, unauthorised subscribe, topic hijack	Abnormal MQTT control-packet ratios, repeated CONNECT without PUBLISH
Spoofing / MITM	ARP spoofing, IP spoofing, data injection, data alteration	TTL anomalies, MAC/IP binding mismatch, duplicate ARP replies, implausible vitals
Malware/Botnet (from D3)	Mirai-style scan + flood	Periodic beaconing, scan-then-flood sequence
Zero-day / Unknown	anything not in the above	High autoencoder reconstruction error, low classifier confidence

6.4 Class imbalance handling

Attack classes in these datasets are wildly imbalanced (benign and MQTT traffic dominate CICIoMT2024; only 12.5% of WUSTL-EHMS is attack traffic). Plan:

1. **Stratified split** 70 / 15 / 15 (train / validation / test) — stratify on the finest label.
2. **Hybrid resampling on the training split only** — never on validation or test: - SMOTE / ADASYN to oversample minority attack classes to a target ratio (not to full balance — full balance causes over-prediction of rare attacks). - Random undersampling of the dominant benign/MQTT class.
3. **Class weights** in XGBoost (`scale_pos_weight`) and Random Forest (`class_weight='balanced_subsample'`) as an alternative to compare against.
4. **Report macro-F1 and per-class recall**, not just accuracy. A 99% accuracy on an imbalanced set can mean 0% recall on the rarest attack.

6.5 Data leakage rules (examiners check this)

- Fit `StandardScaler` , `LabelEncoder` , PCA and SMOTE **only on training data**; transform val/test with the fitted objects.
- Split **by flow, and where possible by capture session**, so packets from the same attack burst do not appear in both train and test.
- Drop identifier columns (`Flow ID` , `Src IP` , `Dst IP` , `Timestamp` , `Src MAC`) from the model input — otherwise the model memorises the attacker's IP and scores 100% on nothing.
- Save the exact split indices to `data/processed/split_indices.npz` so results are reproducible.

7. Feature engineering

7.1 Feature groups (46-feature flow vector)

Group	Example features	Count
Flow identity/meta (used for logging, NOT model input)	flow_id, src_ip, dst_ip, src_port, dst_port, protocol, timestamp	7
Volume	total_fwd_packets, total_bwd_packets, total_fwd_bytes, total_bwd_bytes	4
Packet length statistics	fwd_pkt_len_min/max/mean/std, bwd_pkt_len_min/max/mean/std, pkt_len_var	9
Timing	flow_duration, flow_iat_mean/std/min/max, fwd_iat_mean/std, bwd_iat_mean/std	9
Rate	flow_bytes_per_sec, flow_packets_per_sec, fwd_pkts_per_sec, bwd_pkts_per_sec, down_up_ratio	5
TCP flags	syn_count, ack_count, fin_count, rst_count, psh_count, urg_count, syn_ratio, rst_ratio	8
Header/protocol	fwd_header_len, bwd_header_len, ttl_mean, ttl_std, protocol_onehot(TCP/UDP/ICMP/MQTT)	6
Window/segment	init_win_bytes_fwd, init_win_bytes_bwd, min_seg_size_fwd, act_data_pkt_fwd	4

7.2 IoMT-specific engineered features (your contribution — highlight these)

These do not exist in the standard CIC feature set. Adding them is a real, explainable improvement.

Feature	Definition	Catches
mqtt_connect_ratio	CONNECT packets + total MQTT control packets in the flow	MQTT CONNECT flood
mqtt_pub_no_sub	1 if device publishes to topics it never subscribed to	topic hijack / rogue publisher
device_profile_deviation	Euclidean distance of current flow vector from that device's learned baseline centroid	device compromise
interarrival_regularity	coefficient of variation of packet inter-arrival times	botnet beaconing (too regular) vs human traffic (irregular)
arp_binding_violation	1 if observed MAC→IP pair differs from the registered binding	ARP spoofing / MITM
ttl_deviation	current TTL – device's modal TTL	IP spoofing, injected packets from a different host
vitals_delta_rate	rate of change of the vital carried in the payload (bpm/s, %SpO2/s)	data injection with jumpy values
vitals_physio_violation	1 if vital falls outside physiologically possible range	crude data alteration
payload_entropy	Shannon entropy of payload bytes	encrypted/obfuscated exfiltration
night_hour_flag	1 if flow occurred outside 22:00–06:00 clinical activity window	off-hours anomalous access

7.3 Feature selection pipeline

```
46 raw + 10 engineered = 56 features
├─ Step 1: Remove constant / near-zero variance (VarianceThreshold)
├─ Step 2: Remove one of each pair with |Pearson r| > 0.95
├─ Step 3: Mutual Information ranking vs label → keep top 40
├─ Step 4: Recursive Feature Elimination with CV (RFECV) on XGBoost
├─ Step 5: Cross-check against SHAP global importance
└─
    Final set: ~25–30 features
```

Report the final selected feature list in your report as a table — reducing 56 → ~28 features while holding accuracy is a legitimate result and it cuts inference latency roughly in half.

7.4 Preprocessing sequence (exact order — do not reorder)

- 1. Load raw CSV / parse pcap → dataframe
- 2. Drop identifier columns
- 3. Handle infinities (np.inf → NaN) — CIC flow files contain these in rate columns
- 4. Impute NaN: median for numeric, mode for categorical
- 5. Remove exact duplicate rows
- 6. Clip outliers at the 1st/99th percentile (winsorisation) — do NOT delete them, attacks are outliers
- 7. Encode categorical (protocol , service) with one-hot; encode label with LabelEncoder
- 8. Split into train/val/test (stratified)
- 9. Fit StandardScaler on train → transform all three
- 10. Apply SMOTE/undersampling to train only
- 11. Save scaler + encoder + feature order to models/ (the API must apply the exact same transform)

Critical: persist feature_order.json . At inference time, the column order must match training exactly, or predictions become silently wrong.

8. Machine learning models — model-wise functionality

This is the core of the project. Each model has a distinct, non-overlapping job.

8.1 Model inventory at a glance

ID	Model	Type	Stage	Input	Output	Why chosen
M 1	Random Forest	Supervised ensemble (bagging)	1	28-feature flow vector	benign / malicious + probability	Fast, robust to noise, strong baseline, gives feature importance for free
M 2	XGBoost	Supervised ensemble (boosting)	1 & 2	28-feature flow vector	binary + 6-class attack family + probabilities	Best accuracy/latency trade-off on tabular data; TreeSHAP works exactly on it
M 3	CNN-BiLSTM	Deep sequence model	2 (advanced)	window of 10 consecutive flows	attack family	Captures multi-flow attack patterns (scan → exploit → flood) that single-flow models miss
M 4	Deep Autoencoder	Unsupervised	3	28-feature vector (trained on benign only)	reconstruction error → anomaly score	Detects zero-day attacks; never sees attack data during training
M 5	Isolation Forest	Unsupervised	3 (companion)	same	anomaly score	Cheap second opinion; different inductive bias from the autoencoder
M 6	LSTM Vitals Forecaster	Deep sequence regression	4	last 30 vitals readings per device	predicted next vital + confidence band	Flags data injection/alteration — the medically dangerous attack
M 7	One-Class SVM	Unsupervised	3 (optional/ablation)	same	novelty score	Classical baseline for the novelty-detection comparison table
M 8	Logistic Regression	Supervised linear	baseline	same	binary	Mandatory interpretable baseline for comparison
M 9	SVM (RBF)	Supervised kernel	baseline	same	binary/multiclass	Classical baseline; shows why ensembles win
M 10	Stacking Ensemble	Meta-learner	final	outputs of M1, M2, M3	final label	Squeezes the last 0.5–1% and is a genuine contribution
X 1	SHAP (TreeExplainer + DeepExplainer)	Explainer, not a predictor	5	model + instance	top-k feature contributions	Turns an alert into something a human can act on

8.2 M1 — Random Forest (Stage 1, Binary Detector)

Job: the fast gate. Every flow hits this first. Answer only "benign or malicious".

Why Random Forest here: bagged trees handle mixed-scale tabular features without much tuning, are resistant to the noise in flow statistics, and give a probability estimate you can threshold. Inference is a few hundred microseconds.

Configuration

```
RandomForestClassifier(  
    n_estimators=200,  
    max_depth=25,  
    min_samples_split=5,  
    min_samples_leaf=2,  
    max_features='sqrt',  
    class_weight='balanced_subsample',  
    n_jobs=-1,  
    random_state=42  
)
```

Functionality - Input: scaled 28-dim vector - Output: `{label: 0|1, probability: float}` - Decision rule: `malicious if P(attack) >= 0.5` ; tune the threshold on the validation ROC to hit your target false-positive rate (in a hospital, aim for FPR < 1%) - If `0.35 <= P <= 0.65` → mark as `uncertain` and force the flow through Stage 3 regardless

Expected performance (CICIoMT2024 binary): accuracy 98-99.5%, F1 ≈ 0.98

8.3 M2 — XGBoost (Stage 1 alternate + Stage 2 Multi-class Classifier)

Job: when Stage 1 says malicious, decide *which family* — DDoS, DoS, Recon, MQTT, Spoofing, or Other/Unknown.

Why XGBoost: boosting corrects the errors of previous trees, which matters for the confusable classes (DoS vs DDoS, Recon vs MQTT scan). It also supports `TreeExplainer` in SHAP, so explanations are exact rather than approximated.

Configuration

```
XGBClassifier(  
    objective='multi:softprob',  
    num_class=6,  
    n_estimators=400,  
    max_depth=8,  
    learning_rate=0.08,  
    subsample=0.85,  
    colsample_bytree=0.85,  
    reg_lambda=1.5,  
    reg_alpha=0.5,  
    tree_method='hist',  
    eval_metric='mlogloss',  
    early_stopping_rounds=30,  
    random_state=42  
)
```

Functionality - Input: same 28-dim vector - Output: `{attack_family: str, class_probabilities: {...}, confidence: float}` - If `max(probability) < 0.60` → label as `Unknown/Suspicious` and escalate to Stage 3. This is the handoff that lets the system admit it does not know, instead of guessing.

Known difficulty to report honestly: fine-grained 19-class classification on CICIoMT2024 is much harder than 6-class, because attacks inside the same family (e.g. different Recon variants, MQTT DoS vs MQTT DDoS) produce near-identical traffic statistics. Report both 6-class and 19-class results and discuss the confusion — that discussion is worth more marks than a fake 99.9%.

8.4 M3 — CNN-BiLSTM (Stage 2 advanced, sequence-aware)

Job: detect attacks that only become visible across *multiple consecutive flows*. A port scan is 500 harmless-looking tiny flows. Individually benign; as a sequence, obviously an attack.

Architecture

```
Input: (batch, 10 timesteps, 28 features)  
|  
Conv1D(64, kernel=3, relu) → BatchNorm → MaxPool1D(2)  
|  
Conv1D(128, kernel=3, relu) → BatchNorm → Dropout(0.3)  
|  
Bidirectional(LSTM(64, return_sequences=True))  
|  
Bidirectional(LSTM(32))  
|  
Dense(64, relu) → Dropout(0.4)  
|  
Dense(6, softmax)
```

Why this shape: the Conv1D layers act as learnable local pattern detectors over the flow window (bursts, ramps), and the BiLSTM layers model the order in both directions so the model can use "what happened after" as well as "what happened before" within the window.

Training: Adam (lr 1e-3, ReduceLROnPlateau), categorical cross-entropy, batch 256, 50 epochs with EarlyStopping (patience 8, monitor `val_macro_f1`).

Functionality - Maintains a rolling window of the last 10 flows **per source device** - Emits a family label for the window - Its output is one of the three inputs to the stacking ensemble (M10)

8.5 M4 — Deep Autoencoder (Stage 3, Zero-day Detector)

Job: catch attacks nobody trained it on. This is the answer to the examiner's question "*what if it's a new attack?*"

Key idea: train **only on benign traffic**. The network learns to compress and rebuild normal behaviour. Anything abnormal rebuilds badly, and the reconstruction error spikes.

Architecture

```

Input(28)
→ Dense(24, relu) → Dense(16, relu) → Dense(8, relu) [ENCODER]
→ Dense(4, relu) [BOTTLENECK]
→ Dense(8, relu) → Dense(16, relu) → Dense(24, relu) [DECODER]
→ Dense(28, linear) [RECONSTRUCTION]

```

Training: loss = MSE, Adam lr 1e-3, 100 epochs, benign-only data, 10% held out for threshold calibration.

Threshold selection

```

errors = MSE(benign_validation, reconstructed)
threshold = percentile(errors, 97.5) # tune 95-99 against your FPR budget
anomaly_score = MSE(current_flow, reconstructed) / threshold
is_anomaly = anomaly_score > 1.0

```

Functionality - Runs on **every** flow, including ones Stage 1 called benign — that is the whole point - Output: `{reconstruction_error, anomaly_score, is_anomaly}` - If Stage 1 says benign **and** M4 says anomaly → alert type `POSSIBLE_ZERO_DAY`, severity Medium, requires analyst review - Also serves as **drift detection**: if the average anomaly score on known-benign traffic climbs over weeks, the model needs retraining

8.6 M5 — Isolation Forest (Stage 3 companion)

Job: independent second opinion on anomaly, using a completely different mechanism (random partitioning — anomalies are isolated in fewer splits).

```

IsolationForest(n_estimators=200, contamination=0.02,
max_samples='auto', n_jobs=-1, random_state=42)

```

Functionality: outputs a score in roughly [-0.5, 0.5]. The Stage 3 verdict is `anomaly if (M4 flags) OR (M5 flags)` for maximum recall, or `AND` for maximum precision — make this configurable in `system_config` and show both in your results. Agreement between two independent unsupervised methods is much stronger evidence than either alone.

8.7 M6 — LSTM Vitals Forecaster (Stage 4, Physiological Plausibility)

This is the most distinctive part of your project. No purely network-based IDS can do it.

Job: decide whether the vitals arriving from a device are medically believable, given that patient's recent history.

Two-part check:

(a) Hard physiological bounds (rule layer — instant, no model)

Vital	Absolute possible range	Max plausible change per second
Heart rate	20-250 bpm	5 bpm/s
SpO ₂	50-100 %	2 %/s
Systolic BP	50-250 mmHg	10 mmHg/s
Diastolic BP	30-150 mmHg	8 mmHg/s
Body temperature	30.0-43.0 °C	0.1 °C/s
Respiration rate	4-60 /min	3 /min per s

Any violation → immediate `DATA_INTEGRITY_VIOLATION` alert. A heart rate that jumps from 78 to 180 in one sample is not a patient event, it is an injected packet.

(b) LSTM forecast (learned layer — catches the subtle attacks)

```

Input: (batch, 30 timesteps, 6 vitals)
→ LSTM(64, return_sequences=True) → Dropout(0.2)
→ LSTM(32) → Dropout(0.2)
→ Dense(16, relu)
→ Dense(6, linear) # predicted next reading for all 6 vitals

```

Trained on benign vitals sequences from WUSTL-EHMS-2020 plus your own testbed captures.

```

residual = |actual - predicted|
z = residual / rolling_std(residual_benign)
if z > 3.0 for any vital → suspected data injection

```

Functionality - Runs per device, per patient, on the vitals stream (not the flow stream) - Output: `{predicted_vitals, residuals, z_scores, injection_suspected: bool}` - Correlates with Stage 1/2: if the network layer simultaneously reports ARP anomalies from the same device, confidence in "MITM data injection" becomes very high, and severity is raised to Critical

Why this earns marks: it directly connects a cybersecurity signal to patient safety, which is the actual justification in your problem statement. Without Stage 4, the "Medical" in IoMT is only in the title.

8.8 M7-M9 — Baselines (Logistic Regression, SVM-RBF, One-Class SVM)

Job: these are not for deployment. They exist so your results chapter has a fair comparison table. Every good paper has baselines.

Model	Config	What it proves
Logistic Regression	<code>C=1.0, max_iter=1000, solver='lbfgs'</code>	The problem is not linearly separable — motivates ensembles
SVM (RBF)	<code>C=10, gamma='scale'</code> (subsample to 100k rows or it will not finish)	Kernel methods work but do not scale to millions of flows — motivates trees
One-Class SVM	<code>nu=0.02, kernel='rbf', gamma='scale'</code>	Classical novelty detection underperforms the autoencoder — motivates M4

8.9 M10 — Stacking Ensemble (final decision)

Job: combine M1, M2 and M3 into one final label.



Train the meta-learner using **out-of-fold predictions** (5-fold CV) from the base models, never on their training-set predictions — otherwise the meta-learner learns from overfitted inputs and the stack is worthless.

Expected gain: 0.5–1.5% macro-F1 over the best single model, and — more usefully — better calibrated confidence, which makes the alert severity scoring more trustworthy.

8.10 X1 — SHAP Explainability Layer (Stage 5)

Job: for every alert, produce a human-readable reason.

```
explainer = shap.TreeExplainer(xgb_model) # exact for tree models
shap_values = explainer.shap_values(flow_vector)
top_k = sorted(zip(feature_names, shap_values), key=lambda x: -abs(x[1]))[:5]
```

Stored and displayed as:

Alert #1043 — DDoS (confidence 0.97) on device **Monitor-01** Why the system decided this: 1. **flow_packets_per_sec** = 14,820 → pushed score **+0.41** (normal for this device: 12–60) 2. **syn_ratio** = 0.94 → pushed score **+0.29** (normal: 0.02–0.10) 3. **flow_duration** = 0.004 s → pushed score **+0.18** 4. **fwd_pkt_len_std** = 0.0 → pushed score **+0.11** (identical packet sizes) 5. **down_up_ratio** = 0.001 → pushed score **+0.07**

Also produce, for the report: - **Global SHAP summary (beeswarm) plot** — which features matter overall - **Per-class SHAP bar plots** — which features identify DDoS vs Recon vs Spoofing - **Force plot** for one representative alert of each family

Why it matters in healthcare: an IT admin in a hospital will not disconnect a ventilator's network segment because a black box said "attack". They will act if the system says "this device is emitting 15,000 SYN packets per second with identical sizes". Explainability is what converts a detection into a decision.

9. Model comparison and reporting template

9.1 Results table you must fill in (binary, CICIoMT2024)

Model	Accuracy	Precision	Recall	F1	Macro-F1	ROC-AUC	FPR	Train time	Inference (ms/flow)
Logistic Regression									
SVM (RBF)									
Decision Tree									
Random Forest (M1)									
XGBoost (M2)									
LightGBM									
DNN (3 dense layers)									
CNN-BiLSTM (M3)									
Stacking (M10)									

9.2 Multi-class results (6-class and 19-class) — per-class recall table

Attack family	Support	Precision	Recall	F1
Benign				
DDoS				
DoS				
Recon				
MQTT				
Spoofing				

9.3 Zero-day evaluation protocol (do this — it is the strongest experiment)

- Remove **one entire attack family** (e.g. Spoofing) from the training data of M1/M2.
- Train the autoencoder M4 on benign traffic only.
- Test on the held-out data **including** the removed family.
- Report: M1/M2 recall on the unseen family (will be poor — that is expected and honest), and M4 detection rate on the same family (should be substantially better).
- Repeat, leaving out each family in turn.

This single experiment demonstrates the entire justification for the hybrid architecture, and it is the result to lead with in your paper.

9.4 Cross-dataset generalisation

Train on CICIoMT2024 → test on CICIoT2023 (mapped to common classes) and on your own testbed capture. Accuracy will drop. **Report the drop honestly and explain it** (different testbeds, different device mixes, feature distribution shift). Examiners respect this far more than a suspiciously perfect number.

9.5 Ablation study

Configuration	Macro-F1	Zero-day recall	Notes
Stage 1 only			baseline
Stage 1 + 2			adds attack typing
Stage 1 + 2 + 3			adds zero-day
Stage 1 + 2 + 3 + 4			adds data-injection detection
Full + engineered IoMT features			your contribution

10. Database design

Engine: MySQL 8.0, InnoDB, `utf8mb4_unicode_ci` **Supporting store:** Redis 7 (streams + pub/sub + cache — not persistent state) **Naming convention:** `snake_case` tables (plural), `id` surrogate PK, `*_id` FKs, `created_at` / `updated_at` on every table.

10.1 Entity groups

Group	Tables
Identity & access	<code>roles</code> , <code>users</code> , <code>user_sessions</code> , <code>audit_logs</code>
Clinical & asset inventory	<code>device_types</code> , <code>devices</code> , <code>device_profiles</code> , <code>patients</code> , <code>patient_device_assignments</code> , <code>wards</code>
Telemetry	<code>vital_readings</code> , <code>network_flows</code> , <code>flow_features</code>
Reference data	<code>attack_types</code> , <code>datasets</code> , <code>protocols</code>
ML lifecycle	<code>ml_models</code> , <code>training_runs</code> , <code>model_metrics</code> , <code>feature_importances</code>
Detection & response	<code>detections</code> , <code>explanations</code> , <code>alerts</code> , <code>alert_actions</code> , <code>mitigation_recommendations</code> , <code>notifications</code> , <code>blocklist</code>
System	<code>system_config</code> , <code>thresholds</code> , <code>reports</code>

10.2 ER overview



10.3 Complete table specifications

A. Identity & Access

`roles` — RBAC role definitions

Column	Type	Constraints	Description
id	TINYINT UNSIGNED	PK, AI	
name	VARCHAR(50)	UNIQUE, NOT NULL	admin / security_analyst / clinician / viewer
description	VARCHAR(255)		
permissions	JSON	NOT NULL	e.g. <code>["alerts.read","alerts.ack","models.retrain"]</code>
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

`users`

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AI	
role_id	TINYINT UNSIGNED	FK → roles.id, NOT NULL	
username	VARCHAR(64)	UNIQUE, NOT NULL	login handle
email	VARCHAR(150)	UNIQUE, NOT NULL	
password_hash	VARCHAR(255)	NOT NULL	bcrypt, cost 12
full_name	VARCHAR(150)		
department	VARCHAR(100)		IT Security / ICU / Biomedical Engg
is_active	BOOLEAN	DEFAULT TRUE	

mfa_enabled	BOOLEAN	DEFAULT FALSE	
last_login_at	DATETIME	NULL	
failed_login_count	SMALLINT	DEFAULT 0	lockout after 5
created_at / updated_at	TIMESTAMP		

user_sessions

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AI	
user_id	BIGINT UNSIGNED	FK → users.id	
refresh_token_hash	VARCHAR(255)	NOT NULL	never store raw tokens
ip_address	VARCHAR(45)		IPv6-capable
user_agent	VARCHAR(255)		
issued_at	DATETIME	NOT NULL	
expires_at	DATETIME	NOT NULL	
revoked	BOOLEAN	DEFAULT FALSE	

audit_logs — every privileged action

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AI	
user_id	BIGINT UNSIGNED	FK → users.id, NULL	NULL = system action
action	VARCHAR(100)	NOT NULL	LOGIN, ACK_ALERT, RETRAIN_MODEL, EXPORT_REPORT
entity_type	VARCHAR(50)		alerts / devices / ml_models
entity_id	BIGINT UNSIGNED		
old_value	JSON	NULL	
new_value	JSON	NULL	
ip_address	VARCHAR(45)		
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	INDEX

B. Clinical & Asset Inventory

wards

Column	Type	Constraints	Description
id	INT UNSIGNED	PK, AI	
name	VARCHAR(100)	NOT NULL	ICU-1, General Ward B
floor	VARCHAR(20)		
network_segment	VARCHAR(50)		VLAN / subnet, e.g. 192.168.20.0/24
criticality	ENUM('low','medium','high','critical')	DEFAULT 'medium'	drives alert severity

device_types

Column	Type	Constraints	Description
id	INT UNSIGNED	PK, AI	
type_name	VARCHAR(100)	UNIQUE, NOT NULL	Patient Monitor, Infusion Pump, Ventilator, Pulse Oximeter, ECG, Smart Bed
category	ENUM('sensor','actuator','gateway','hybrid')	NOT NULL	maps to your block diagram
is_life_critical	BOOLEAN	DEFAULT FALSE	ventilator/pump = TRUE → never auto-block
default_protocols	JSON		["MQTT", "TCP"]
expected_data_rate_kbps	DECIMAL(8,2)		baseline for profiling

devices

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AI	
device_uid	VARCHAR(64)	UNIQUE, NOT NULL	e.g. <code>MON-ICU1-003</code>
device_type_id	INT UNSIGNED	FK → device_types.id	
ward_id	INT UNSIGNED	FK → wards.id, NULL	
manufacturer	VARCHAR(100)		
model_number	VARCHAR(100)		
firmware_version	VARCHAR(50)		used for vulnerability correlation
ip_address	VARCHAR(45)	INDEX	
mac_address	VARCHAR(17)	UNIQUE, INDEX	ground truth for ARP-spoof detection
registered_mac_ip_binding	VARCHAR(64)		<code>MAC IP</code> snapshot
mqtt_client_id	VARCHAR(100)		
status	ENUM('online','offline','quarantined','maintenance')	DEFAULT 'offline'	

trust_score	DECIMAL(5,2)	DEFAULT 100.00	decays on detections, recovers over time
last_seen_at	DATETIME	INDEX	
created_at / updated_at	TIMESTAMP		

device_profiles — learned behavioural baseline (one row per device)

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AI	
device_id	BIGINT UNSIGNED	FK → devices.id, UNIQUE	
baseline_vector	JSON	NOT NULL	mean of each of the 28 features
baseline_std	JSON	NOT NULL	std of each feature
typical_ports	JSON		[1883, 8883]
typical_peers	JSON		IPs it normally talks to
modal_ttl	SMALLINT		for TTL-deviation feature
avg_pkts_per_sec	DECIMAL(10,3)		
active_hours	JSON		{"start": "06:00", "end": "22:00"}
samples_used	INT UNSIGNED		how many benign flows built this
profile_version	INT	DEFAULT 1	
last_recomputed_at	DATETIME		rebuild weekly

patients — pseudonymised only; **never store real identifiers in a student project**

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AI	
patient_code	VARCHAR(32)	UNIQUE, NOT NULL	PT-0001 — pseudonym, no name
age_band	VARCHAR(16)		30-39 (banded, not exact)
sex	ENUM('M','F','O','U')		
ward_id	INT UNSIGNED	FK → wards.id	
admitted_at	DATETIME		
discharged_at	DATETIME	NULL	
baseline_hr_min / baseline_hr_max	SMALLINT		personalised plausibility bounds
baseline_spo2_min	TINYINT		
notes	VARCHAR(255)		no clinical detail

patient_device_assignments

Column	Type	Constraints
id	BIGINT UNSIGNED	PK, AI
patient_id	BIGINT UNSIGNED	FK → patients.id
device_id	BIGINT UNSIGNED	FK → devices.id
assigned_at	DATETIME	NOT NULL
released_at	DATETIME	NULL
		UNIQUE(device_id, assigned_at)

C. Telemetry

vital_readings — the physiological stream (Stage 4 input). High volume → partition by month.

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AI	
device_id	BIGINT UNSIGNED	FK → devices.id, INDEX	
patient_id	BIGINT UNSIGNED	FK → patients.id, NULL	
recorded_at	DATETIME(3)	NOT NULL, INDEX	millisecond precision
heart_rate	SMALLINT	NULL	bpm
spo2	TINYINT	NULL	%
systolic_bp	SMALLINT	NULL	mmHg
diastolic_bp	SMALLINT	NULL	mmHg
body_temp	DECIMAL(4,2)	NULL	°C
respiration_rate	TINYINT	NULL	breaths/min
raw_payload	JSON	NULL	as received
is_plausible	BOOLEAN	DEFAULT TRUE	rule-layer verdict
predicted_values	JSON	NULL	M6 forecast
residual_zscore	DECIMAL(6,3)	NULL	max z across vitals
injection_suspected	BOOLEAN	DEFAULT FALSE	INDEX
			INDEX(device_id, recorded_at)

`network_flows` — one row per assembled bidirectional flow. Highest-volume table.

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AI	
flow_uid	CHAR(36)	UNIQUE, NOT NULL	UUID
device_id	BIGINT UNSIGNED	FK → devices.id, NULL, INDEX	resolved by IP/MAC lookup
src_ip	VARCHAR(45)	NOT NULL, INDEX	
dst_ip	VARCHAR(45)	NOT NULL, INDEX	
src_port	INT UNSIGNED		
dst_port	INT UNSIGNED	INDEX	
protocol	ENUM('TCP','UDP','ICMP','MQTT','HTTP','BLE','OTHER')	NOT NULL	
src_mac	VARCHAR(17)		for ARP-spoof check
dst_mac	VARCHAR(17)		
flow_start	DATETIME(3)	NOT NULL, INDEX	
flow_end	DATETIME(3)	NOT NULL	
duration_ms	INT UNSIGNED		
total_fwd_packets	INT UNSIGNED		
total_bwd_packets	INT UNSIGNED		
total_fwd_bytes	BIGINT UNSIGNED		
total_bwd_bytes	BIGINT UNSIGNED		
capture_source	ENUM('live','pcap','dataset','simulated')	DEFAULT 'live'	
pcap_reference	VARCHAR(255)	NULL	path to evidence file
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

`flow_features` — the model input vector (1:1 with `network_flows`)

Column	Type	Description
id	BIGINT UNSIGNED PK AI	
flow_id	BIGINT UNSIGNED, FK → network_flows.id, UNIQUE	
feature_vector	JSON, NOT NULL	ordered array of the final ~28 values
feature_set_version	VARCHAR(20), NOT NULL	must match the model's expected version
flow_bytes_per_sec	DECIMAL(14,3)	denormalised for fast dashboard queries
flow_packets_per_sec	DECIMAL(14,3)	
syn_ratio	DECIMAL(5,4)	
rst_ratio	DECIMAL(5,4)	
down_up_ratio	DECIMAL(10,4)	
pkt_len_mean	DECIMAL(10,3)	
pkt_len_std	DECIMAL(10,3)	
iat_mean_ms	DECIMAL(12,3)	
ttl_deviation	SMALLINT	
payload_entropy	DECIMAL(6,4)	
mqtt_connect_ratio	DECIMAL(5,4)	
device_profile_deviation	DECIMAL(10,4)	
arp_binding_violation	BOOLEAN	
created_at	TIMESTAMP	

D. Reference Data

`attack_types`

Column	Type	Description
id	INT UNSIGNED PK AI	
code	VARCHAR(40) UNIQUE	DDOS_ICMP_FLOOD , MQTT_CONNECT_FLOOD , ARP_SPOOF , DATA_INJECTION
family	ENUM('BENIGN','DDOS','DOS','RECON','MQTT','SPOOFING','MALWARE','UNKNOWN')	maps to M2's 6-class output
display_name	VARCHAR(100)	"ICMP Flood (DDoS)"
description	TEXT	
default_severity	ENUM('info','low','medium','high','critical')	
mitre_technique	VARCHAR(20)	e.g. T1498 — cite ATT&CK, examiners like this
typical_indicators	JSON	["high pps","uniform pkt size"]
recommended_action	TEXT	

`datasets`

Column	Type	Description
--------	------	-------------

id	INT UNSIGNED PK AI	
name	VARCHAR(100) UNIQUE	CICIoMT2024
version	VARCHAR(20)	
source_url	VARCHAR(255)	
total_records	BIGINT UNSIGNED	
num_features	SMALLINT	
num_classes	SMALLINT	
benign_ratio	DECIMAL(5,4)	
citation	TEXT	full reference string
notes	TEXT	

`protocols` — small lookup used by the dashboard filters (id, name, default_port, is_medical).

E. ML Lifecycle

`ml_models` — the model registry

Column	Type	Description
id	INT UNSIGNED PK AI	
model_code	VARCHAR(40) UNIQUE	M1_RF_BINARY, M2_XGB_MULTICLASS, M4_AUTOENCODER, M6_VITALS_LSTM
display_name	VARCHAR(100)	
stage	TINYINT	1-5, which pipeline stage it serves
algorithm	VARCHAR(60)	RandomForest, XGBoost, CNN-BiLSTM, Autoencoder, LSTM
task_type	ENUM('binary','multiclass','anomaly','regression','meta')	
version	VARCHAR(20) NOT NULL	v1.0.3
artifact_path	VARCHAR(255) NOT NULL	models/binary_rf_v1.pkl
scaler_path	VARCHAR(255)	
feature_set_version	VARCHAR(20)	must match flow_features
hyperparameters	JSON	full config used
input_dim	SMALLINT	
output_classes	JSON	ordered class labels
threshold	DECIMAL(6,4)	decision / anomaly threshold
is_active	BOOLEAN DEFAULT FALSE	only one active per stage
trained_at	DATETIME	
trained_by	BIGINT UNSIGNED FK → users.id	
created_at	TIMESTAMP	

`training_runs`

Column	Type	Description
id	BIGINT UNSIGNED PK AI	
model_id	INT UNSIGNED FK → ml_models.id	
dataset_id	INT UNSIGNED FK → datasets.id	
run_uid	CHAR(36) UNIQUE	
train_size / val_size / test_size	INT UNSIGNED	
resampling_method	VARCHAR(50)	SMOTE / ADASYN / none / class_weight
epochs	INT	NULL for tree models
batch_size	INT	
duration_seconds	INT UNSIGNED	
hardware	VARCHAR(100)	"Colab T4 GPU"
status	ENUM('running','completed','failed')	
loss_curve	JSON	per-epoch train/val loss, for the report plots
notes	TEXT	
started_at / finished_at	DATETIME	

`model_metrics` — one row per (run, split, class)

Column	Type	Description
id	BIGINT UNSIGNED PK AI	
training_run_id	BIGINT UNSIGNED FK	
split	ENUM('train','validation','test','cross_dataset')	
class_label	VARCHAR(50)	<code>__overall__</code> or a class name
accuracy / precision / recall / f1_score / roc_auc / pr_auc	DECIMAL(7,6)	
false_positive_rate / false_negative_rate	DECIMAL(7,6)	

support	INT UNSIGNED	
confusion_matrix	JSON	full matrix for <code>__overall__</code>
avg_inference_ms	DECIMAL(8,3)	
created_at	TIMESTAMP	

feature_importances

Column	Type	Description
id	BIGINT UNSIGNED PK AI	
model_id	INT UNSIGNED FK	
feature_name	VARCHAR(80)	
importance_gain	DECIMAL(10,8)	native model importance
shap_mean_abs	DECIMAL(10,8)	global SHAP importance
rank	SMALLINT	

F. Detection & Response

detections — one row per flow per pipeline execution

Column	Type	Description
id	BIGINT UNSIGNED PK AI	
flow_id	BIGINT UNSIGNED FK → network_flows.id, INDEX	
device_id	BIGINT UNSIGNED FK → devices.id, INDEX	
stage1_model_id	INT UNSIGNED FK → ml_models.id	
stage1_label	ENUM('benign','malicious') NOT NULL	
stage1_probability	DECIMAL(6,5)	
stage2_model_id	INT UNSIGNED FK, NULL	
stage2_attack_type_id	INT UNSIGNED FK → attack_types.id, NULL	
stage2_confidence	DECIMAL(6,5)	
stage2_class_probs	JSON	full distribution
stage3_model_id	INT UNSIGNED FK, NULL	
stage3_reconstruction_error	DECIMAL(12,8)	
stage3_anomaly_score	DECIMAL(10,6)	error ÷ threshold
stage3_is_anomaly	BOOLEAN	
stage4_injection_suspected	BOOLEAN	
stage4_max_zscore	DECIMAL(8,4)	
final_verdict	ENUM('benign','known_attack','zero_day_suspect','data_integrity','uncertain') NOT NULL, INDEX	
final_confidence	DECIMAL(6,5)	from the stacking meta-learner
severity	ENUM('info','low','medium','high','critical') NOT NULL, INDEX	
inference_latency_ms	DECIMAL(8,3)	
detected_at	DATETIME(3) NOT NULL, INDEX	
		INDEX(device_id, detected_at), INDEX(final_verdict, severity)

explanations — SHAP output per detection

Column	Type	Description
id	BIGINT UNSIGNED PK AI	
detection_id	BIGINT UNSIGNED FK → detections.id, UNIQUE	
method	ENUM('SHAP','LIME') DEFAULT 'SHAP'	
base_value	DECIMAL(12,8)	model expected value
top_features	JSON NOT NULL	[{"feature": "flow_packets_per_sec", "value": 14820, "shap": 0.41, "normal_range": "12-60"}, ...]
narrative	TEXT	auto-generated plain-English sentence shown in the UI
created_at	TIMESTAMP	

alerts — a detection promoted to something a human must handle

Column	Type	Description
id	BIGINT UNSIGNED PK AI	
alert_uid	VARCHAR(20) UNIQUE	ALT-2026-001043
detection_id	BIGINT UNSIGNED FK → detections.id	
device_id	BIGINT UNSIGNED FK → devices.id, INDEX	
patient_id	BIGINT UNSIGNED FK → patients.id, NULL	who is at risk
attack_type_id	INT UNSIGNED FK → attack_types.id, NULL	
title	VARCHAR(200) NOT NULL	
description	TEXT	

severity	ENUM('info','low','medium','high','critical') NOT NULL, INDEX	
risk_score	DECIMAL(5,2)	0-100, see §14.2
status	ENUM('new','acknowledged','investigating','resolved','false_positive') DEFAULT 'new', INDEX	
assigned_to	BIGINT UNSIGNED FK → users.id, NULL	
correlated_alert_id	BIGINT UNSIGNED FK → alerts.id, NULL	groups a flood into one incident
occurrence_count	INT UNSIGNED DEFAULT 1	dedupe counter
first_seen_at / last_seen_at	DATETIME(3)	
acknowledged_at / resolved_at	DATETIME NULL	
resolution_notes	TEXT	
created_at / updated_at	TIMESTAMP	

alert_actions — full audit trail of the alert lifecycle

Column	Type	Description
id	BIGINT UNSIGNED PK AI	
alert_id	BIGINT UNSIGNED FK → alerts.id	
user_id	BIGINT UNSIGNED FK → users.id, NULL	
action	ENUM('created','acknowledged','assigned','commented','escalated','mitigated','resolved','marked_false_positive','reopened')	
comment	TEXT	
previous_status / new_status	VARCHAR(30)	
created_at	TIMESTAMP, INDEX	

mitigation_recommendations — advice, never automatic action on life-critical devices

Column	Type	Description
id	BIGINT UNSIGNED PK AI	
alert_id	BIGINT UNSIGNED FK → alerts.id	
recommendation	VARCHAR(255) NOT NULL	"Rate-limit source 192.168.20.55 at the gateway"
action_type	ENUM('rate_limit','isolate_source','block_ip','revoke_mqtt_client','notify_biomed','force_reauth','manual_review')	
target	VARCHAR(100)	IP / MAC / client_id
is_automatable	BOOLEAN DEFAULT FALSE	FALSE whenever the device is life-critical
requires_approval	BOOLEAN DEFAULT TRUE	
applied	BOOLEAN DEFAULT FALSE	
applied_by	BIGINT UNSIGNED FK → users.id, NULL	
applied_at	DATETIME NULL	

notifications

Column	Type	Description
id	BIGINT UNSIGNED PK AI	
alert_id	BIGINT UNSIGNED FK	
channel	ENUM('email','sms','telegram','webhook','websocket')	
recipient	VARCHAR(150)	
payload	JSON	
status	ENUM('pending','sent','failed') DEFAULT 'pending'	
retry_count	TINYINT DEFAULT 0	
error_message	VARCHAR(255)	
sent_at	DATETIME NULL	

blocklist — sources under restriction (never contains a life-critical device)

Column	Type	Description
id	BIGINT UNSIGNED PK AI	
entry_type	ENUM('ip','mac','mqtt_client')	
value	VARCHAR(100) NOT NULL, INDEX	
reason	VARCHAR(255)	
alert_id	BIGINT UNSIGNED FK, NULL	
added_by	BIGINT UNSIGNED FK → users.id	
expires_at	DATETIME NULL	NULL = permanent
is_active	BOOLEAN DEFAULT TRUE	
created_at	TIMESTAMP	

G. System

system_config — runtime tunables (avoid hardcoding)

Column	Type	Description
id	INT UNSIGNED PK AI	
config_key	VARCHAR(80) UNIQUE	stage1_threshold , autoencoder_percentile , stage3_combine_mode
config_value	VARCHAR(255)	
value_type	ENUM('string','int','float','bool','json')	
description	VARCHAR(255)	
updated_by	BIGINT UNSIGNED FK → users.id	
updated_at	TIMESTAMP	

thresholds — per-device-type or per-vital limits used by Stage 4's rule layer

Column	Type	Description
id	INT UNSIGNED PK AI	
scope	ENUM('global','device_type','device','patient')	
scope_id	BIGINT UNSIGNED NULL	
metric	VARCHAR(50)	heart_rate , spo2 , packets_per_sec
min_value / max_value	DECIMAL(12,4)	
max_delta_per_sec	DECIMAL(12,4)	
is_active	BOOLEAN DEFAULT TRUE	

reports

Column	Type	Description
id	BIGINT UNSIGNED PK AI	
report_type	ENUM('daily','weekly','incident','compliance','custom')	
title	VARCHAR(200)	
period_start / period_end	DATETIME	
filters	JSON	
summary_stats	JSON	totals, top attacks, top devices, MTTD
file_path	VARCHAR(255)	generated PDF/CSV
generated_by	BIGINT UNSIGNED FK → users.id	
created_at	TIMESTAMP	

10.4 Sample DDL (copy-paste ready for the critical tables)

```
CREATE DATABASE iomt_ids
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE iomt_ids;

CREATE TABLE roles (
  id          TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  name        VARCHAR(50)  NOT NULL,
  description  VARCHAR(255) DEFAULT NULL,
  permissions  JSON        NOT NULL,
  created_at   TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_roles_name (name)
) ENGINE=InnoDB;

CREATE TABLE users (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  role_id     TINYINT UNSIGNED NOT NULL,
  username     VARCHAR(64)  NOT NULL,
  email        VARCHAR(150) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  full_name    VARCHAR(150) DEFAULT NULL,
  department   VARCHAR(100) DEFAULT NULL,
  is_active    BOOLEAN      NOT NULL DEFAULT TRUE,
  mfa_enabled  BOOLEAN      NOT NULL DEFAULT FALSE,
  last_login_at DATETIME    DEFAULT NULL,
  failed_login_count SMALLINT NOT NULL DEFAULT 0,
  created_at   TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at   TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_users_username (username),
  UNIQUE KEY uq_users_email (email),
  CONSTRAINT fk_users_role FOREIGN KEY (role_id) REFERENCES roles(id)
) ENGINE=InnoDB;

CREATE TABLE device_types (
  id          INT UNSIGNED NOT NULL AUTO_INCREMENT,
  type_name    VARCHAR(100) NOT NULL,
  category     ENUM('sensor','actuator','gateway','hybrid') NOT NULL,
  is_life_critical BOOLEAN NOT NULL DEFAULT FALSE,
  default_protocols JSON DEFAULT NULL,
  expected_data_rate_kbps DECIMAL(8,2) DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_device_type_name (type_name)
) ENGINE=InnoDB;

CREATE TABLE devices (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  device_uid   VARCHAR(64) NOT NULL,
  device_type_id INT UNSIGNED NOT NULL,
  ward_id      INT UNSIGNED DEFAULT NULL,
  manufacturer VARCHAR(100) DEFAULT NULL,
  model_number  VARCHAR(100) DEFAULT NULL,
  firmware_version VARCHAR(50) DEFAULT NULL,
  ip_address    VARCHAR(45)  DEFAULT NULL,
  mac_address   VARCHAR(17)  DEFAULT NULL,
  mqtt_client_id VARCHAR(100) DEFAULT NULL,
  status        ENUM('online','offline','quarantined','maintenance')
              NOT NULL DEFAULT 'offline',
  trust_score   DECIMAL(5,2) NOT NULL DEFAULT 100.00,
```

```

last_seen_at    DATETIME DEFAULT NULL,
created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
                ON UPDATE CURRENT_TIMESTAMP,

PRIMARY KEY (id),
UNIQUE KEY uq_device_uid (device_uid),
UNIQUE KEY uq_device_mac (mac_address),
KEY idx_device_ip (ip_address),
KEY idx_device_last_seen (last_seen_at),
CONSTRAINT fk_devices_type FOREIGN KEY (device_type_id)
    REFERENCES device_types(id),
CONSTRAINT fk_devices_ward FOREIGN KEY (ward_id) REFERENCES wards(id)
) ENGINE=InnoDB;

CREATE TABLE network_flows (
  id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  flow_uid        CHAR(36) NOT NULL,
  device_id       BIGINT UNSIGNED DEFAULT NULL,
  src_ip          VARCHAR(45) NOT NULL,
  dst_ip          VARCHAR(45) NOT NULL,
  src_port        INT UNSIGNED DEFAULT NULL,
  dst_port        INT UNSIGNED DEFAULT NULL,
  protocol        ENUM('TCP','UDP','ICMP','MQTT','HTTP','BLE','OTHER') NOT NULL,
  src_mac         VARCHAR(17) DEFAULT NULL,
  dst_mac         VARCHAR(17) DEFAULT NULL,
  flow_start      DATETIME(3) NOT NULL,
  flow_end        DATETIME(3) NOT NULL,
  duration_ms     INT UNSIGNED DEFAULT NULL,
  total_fwd_packets INT UNSIGNED DEFAULT 0,
  total_bwd_packets INT UNSIGNED DEFAULT 0,
  total_fwd_bytes  BIGINT UNSIGNED DEFAULT 0,
  total_bwd_bytes  BIGINT UNSIGNED DEFAULT 0,
  capture_source   ENUM('live','pcap','dataset','simulated') NOT NULL DEFAULT 'live',
  pcap_reference   VARCHAR(255) DEFAULT NULL,
  created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_flow_uid (flow_uid),
  KEY idx_flow_device_time (device_id, flow_start),
  KEY idx_flow_src (src_ip),
  KEY idx_flow_dst (dst_ip),
  CONSTRAINT fk_flows_device FOREIGN KEY (device_id) REFERENCES devices(id)
) ENGINE=InnoDB;

CREATE TABLE detections (
  id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  flow_id         BIGINT UNSIGNED NOT NULL,
  device_id       BIGINT UNSIGNED DEFAULT NULL,
  stage1_model_id INT UNSIGNED DEFAULT NULL,
  stage1_label    ENUM('benign','malicious') NOT NULL,
  stage1_probability DECIMAL(6,5) DEFAULT NULL,
  stage2_model_id INT UNSIGNED DEFAULT NULL,
  stage2_attack_type_id INT UNSIGNED DEFAULT NULL,
  stage2_confidence DECIMAL(6,5) DEFAULT NULL,
  stage2_class_probs JSON DEFAULT NULL,
  stage3_model_id INT UNSIGNED DEFAULT NULL,
  stage3_reconstruction_error DECIMAL(12,8) DEFAULT NULL,
  stage3_anomaly_score DECIMAL(10,6) DEFAULT NULL,
  stage3_is_anomaly  BOOLEAN NOT NULL DEFAULT FALSE,
  stage4_injection_suspected BOOLEAN NOT NULL DEFAULT FALSE,
  stage4_max_zscore  DECIMAL(8,4) DEFAULT NULL,
  final_verdict     ENUM('benign','known_attack','zero_day_suspect',
    'data_integrity','uncertain') NOT NULL,
  final_confidence  DECIMAL(6,5) DEFAULT NULL,
  severity          ENUM('info','low','medium','high','critical')
    NOT NULL DEFAULT 'info',
  inference_latency_ms DECIMAL(8,3) DEFAULT NULL,
  detected_at       DATETIME(3) NOT NULL,
  PRIMARY KEY (id),
  KEY idx_det_flow (flow_id),
  KEY idx_det_device_time (device_id, detected_at),
  KEY idx_det_verdict (final_verdict, severity),
  CONSTRAINT fk_det_flow FOREIGN KEY (flow_id) REFERENCES network_flows(id),
  CONSTRAINT fk_det_attack FOREIGN KEY (stage2_attack_type_id)
    REFERENCES attack_types(id)
) ENGINE=InnoDB;

CREATE TABLE alerts (
  id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  alert_uid       VARCHAR(20) NOT NULL,
  detection_id    BIGINT UNSIGNED NOT NULL,
  device_id       BIGINT UNSIGNED DEFAULT NULL,
  patient_id      BIGINT UNSIGNED DEFAULT NULL,
  attack_type_id  INT UNSIGNED DEFAULT NULL,
  title           VARCHAR(200) NOT NULL,
  description      TEXT,
  severity        ENUM('info','low','medium','high','critical') NOT NULL,
  risk_score      DECIMAL(5,2) DEFAULT NULL,
  status          ENUM('new','acknowledged','investigating','resolved',
    'false_positive') NOT NULL DEFAULT 'new',
  assigned_to     BIGINT UNSIGNED DEFAULT NULL,
  correlated_alert_id BIGINT UNSIGNED DEFAULT NULL,
  occurrence_count INT UNSIGNED NOT NULL DEFAULT 1,
  first_seen_at   DATETIME(3) NOT NULL,
  last_seen_at    DATETIME(3) NOT NULL,
  acknowledged_at DATETIME DEFAULT NULL,
  resolved_at     DATETIME DEFAULT NULL,
  resolution_notes TEXT,
  created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,

PRIMARY KEY (id),
UNIQUE KEY uq_alert_uid (alert_uid),
KEY idx_alert_status_sev (status, severity),
KEY idx_alert_device (device_id),
CONSTRAINT fk_alert_detection FOREIGN KEY (detection_id)
    REFERENCES detections(id),
CONSTRAINT fk_alert_user FOREIGN KEY (assigned_to) REFERENCES users(id)
) ENGINE=InnoDB;

```

10.5 Performance notes (mention these in the report — they show maturity)

- **Partition** `network_flows` , `flow_features` , `detections` and `vital_readings` by `RANGE (TO_DAYS(...))` monthly. Volume is the main scaling risk.
- **Batch inserts** — write flows in batches of 500 via `executemany` , never one INSERT per packet.
- **Retention policy** — keep raw flows 30 days, detections 1 year, alerts and audit logs 3 years (aligns with healthcare record-keeping expectations).
- **Redis** holds the last 5 minutes of flows for the live dashboard so the UI never queries MySQL on every tick.
- **Read model** — create a summary table `hourly_stats(hour, device_id, total_flows, malicious_flows, top_attack_type_id)` refreshed by a scheduled job, so dashboard charts do not aggregate millions of rows on every page load.

11. Module-wise functionality

The system is 8 modules. Each maps to a folder in §5.3 and can be built and tested independently.

Module 1 — Device Simulator & Gateway

Purpose: produce realistic IoT traffic (benign + attack) so you can demo without a hospital. **Functionality:** - `device_monitor.py` publishes vitals (HR, SpO₂, BP, temp) to MQTT every 1–2 s with natural variation - `device_pump.py`, ECG sim, etc. publish their own patterns - Gateway subscribes to all topics, timestamps, normalises, forwards vitals to the backend `/api/v1/vitals` and mirrors traffic to the sniffer - Attacker scripts inject DoS/DDoS/recon/MITM/MQTT-flood on demand via a simple CLI so you can trigger each attack live in the demo

Module 2 — Capture & Feature Extraction

Purpose: turn raw packets into the model's input vector. **Functionality:** - `sniffer.py` captures on the interface (Scapy `AsyncSniffer`) - `flow_assembler.py` groups packets into bidirectional flows with a 120 s timeout / 15 s idle timeout - `feature_extractor.py` computes the 46 statistical + 10 engineered features, applies the saved scaler and feature order, pushes the vector to a Redis stream `flows:incoming` - Also handles offline mode: read a `.pcap` or dataset CSV and replay it through the same path (so live and offline use identical code)

Module 3 — Detection Engine

Purpose: run Stages 1–5. **Functionality:** - `detection_worker.py` consumes `flows:incoming`, loads all models once at startup, runs the pipeline, writes `detections` + `explanations`, and — if malicious/anomalous — calls the Alert Service - Loads the **active** model per stage from `ml_models` (so you can swap models without code changes) - Records `inference_latency_ms` for every flow (needed for your results)

Module 4 — Vitals Plausibility Service

Purpose: Stage 4. **Functionality:** - Consumes the vitals stream, runs the rule bounds + LSTM forecast per device - Writes `vital_readings` with `is_plausible`, `residual_zscore`, `injection_suspected` - Raises `DATA_INTEGRITY_VIOLATION` alerts and correlates with any concurrent network detection on the same device

Module 5 — Alert & Notification Service

Purpose: turn detections into actionable, de-duplicated, prioritised alerts. **Functionality:** - Deduplicates: a 10,000-packet flood becomes **one** alert with `occurrence_count`, not 10,000 alerts - Computes `risk_score` (see §14.2), assigns `severity` - Generates `mitigation_recommendations` (advisory) - Pushes to WebSocket, sends email/Telegram for high/critical - Manages the alert lifecycle (new → acknowledged → investigating → resolved / false_positive) with a full `alert_actions` trail

Module 6 — Backend API

Purpose: everything the frontend and external tools talk to. Full endpoint list in §12.

Module 7 — Dashboard (Frontend)

Purpose: the human interface. Pages in §13.

Module 8 — Model Training & Evaluation Pipeline

Purpose: the offline ML work (the notebooks + `ml/` scripts). Produces the serialised models, metrics, and plots that Module 3 loads and that your report presents.

12. API specification (REST + WebSocket)

Base path `/api/v1`. Auth: `Authorization: Bearer <JWT>`. All list endpoints paginate (`?page=&size=`) and filter.

Auth

Method	Path	Body / Params	Returns
POST	<code>/auth/register</code>	username, email, password, role	user
POST	<code>/auth/login</code>	username, password	access + refresh token
POST	<code>/auth/refresh</code>	refresh_token	new access token
POST	<code>/auth/logout</code>	—	204
GET	<code>/auth/me</code>	—	current user + role

Devices

Method	Path	Purpose
GET	<code>/devices</code>	list + filter by ward/type/status
POST	<code>/devices</code>	register a device
GET	<code>/devices/{id}</code>	detail incl. profile + trust_score
PATCH	<code>/devices/{id}</code>	update / change status (quarantine)
GET	<code>/devices/{id}/flows</code>	recent flows for the device
GET	<code>/devices/{id}/vitals</code>	recent vitals
GET	<code>/devices/{id}/timeline</code>	detections + alerts timeline

Flows & Detections

Method	Path	Purpose
GET	<code>/flows</code>	paginated flows, filter by time/protocol/device
POST	<code>/detect</code>	core: submit a feature vector → full pipeline verdict (used by worker + for manual testing)
GET	<code>/detections</code>	filter by verdict/severity/device/time
GET	<code>/detections/{id}</code>	detail incl. all five stages + explanation

Alerts

Method	Path	Purpose
GET	/alerts	filter by status/severity/device/time
GET	/alerts/{id}	full detail + explanation + recommendations + action history
POST	/alerts/{id}/acknowledge	ack
POST	/alerts/{id}/assign	assign to user
POST	/alerts/{id}/resolve	resolve + notes
POST	/alerts/{id}/false-positive	mark FP (feeds retraining)
POST	/alerts/{id}/comment	add comment

Models

Method	Path	Purpose
GET	/models	registry list
GET	/models/{id}/metrics	metrics across runs
POST	/models/{id}/activate	make this the active model for its stage
POST	/models/retrain	trigger a retraining run (admin)
GET	/models/{id}/feature-importance	for the dashboard SHAP page

Reports & Stats

Method	Path	Purpose
GET	/stats/overview	dashboard KPI cards
GET	/stats/attack-distribution	pie/bar data
GET	/stats/traffic-timeline	line chart data
POST	/reports/generate	build a PDF/CSV report
GET	/reports/{id}/download	download it

Example /detect contract

```
// POST /api/v1/detect
{
  "flow_uid": "b2e1...",
  "device_id": 12,
  "feature_vector": [0.12, 3.4, ...], // length must equal model input_dim
  "feature_set_version": "v1"
}
// 200 OK
{
  "detection_id": 88213,
  "final_verdict": "known attack",
  "attack_family": "DDoS",
  "severity": "high",
  "final_confidence": 0.971,
  "stages": {
    "stage1": {"label": "malicious", "probability": 0.994},
    "stage2": {"family": "DDoS", "confidence": 0.971},
    "stage3": {"anomaly_score": 2.14, "is_anomaly": true},
    "stage4": {"injection_suspected": false}
  },
  "explanation": {
    "top_features": [
      {"feature": "flow_packets_per_sec", "value": 14820, "shap": 0.41, "normal_range": "12-60"}
    ],
    "narrative": "Flagged as DDoS: packet rate ~250x this device's normal, near-uniform packet sizes, SYN ratio 0.94."
  },
  "inference_latency_ms": 7.3
}
```

WebSocket

- ws://host/ws/alerts — server pushes each new alert as JSON the instant it is created (live console)
- ws://host/ws/traffic — throttled live flow/detection feed for the dashboard traffic graph

13. Dashboard pages (frontend functionality)

Page	What it shows	Key components
Overview	KPI cards (flows today, attacks today, active alerts, devices online, mean-time-to-detect), live traffic line chart, attack-distribution donut, latest 10 alerts	WebSocket-driven, auto-refresh
Live Monitor	Real-time scrolling table of flows with verdict colour-coding; red row on attack; live pps gauge per device	subscribes to /ws/traffic
Alerts Console	Filterable alert table; click → detail drawer with SHAP explanation, recommendations, action buttons, correlated alerts	the operational heart of the UI
Devices	Inventory grid; per device: type, ward, status, trust_score, last seen, mini traffic sparkline; quarantine button	
Device Detail	Timeline of that device's flows, detections, vitals; vitals chart with predicted vs actual band (shows injection visually)	
Models	Registry, per-model metrics, confusion matrices, ROC curves, global SHAP beeswarm, activate/retrain controls	
Reports	Generate/download daily/weekly/incident reports	
Attack Simulator (demo-only, admin)	Buttons to launch each attack against a chosen device so you can demo detection live	wraps the attacker scripts

14. Detection logic details

14.1 Final verdict decision table

Stage 1	Stage 2 conf	Stage 3 anomaly	Stage 4 injection	→ Final verdict	Severity
benign	—	no	no	benign	info
benign	—	yes	no	zero_day_suspect	medium
benign	—	no	yes	data_integrity	high
malicious	≥0.60	—	no	known_attack (family)	family default
malicious	<0.60	yes	—	zero_day_suspect	medium-high
malicious	≥0.60	—	yes	known_attack + data_integrity	critical
uncertain (0.35–0.65)	—	yes	—	zero_day_suspect	medium

14.2 Risk score (0-100)

```
risk = 100 * (
    0.35 * severity_weight      # info .1 ... critical 1.0
    + 0.25 * final_confidence
    + 0.20 * device_criticality  # ward + life-critical flag
    + 0.10 * anomaly_score_norm
    + 0.10 * patient_at_risk    # 1 if a patient is assigned to the device
)
```

This is what sorts the alert queue. A DDoS on a spare pulse-oximeter in storage ranks far below a data-injection on a ventilator attached to a patient — which is exactly the clinical prioritisation you want to demonstrate.

14.3 Safety rule (state this explicitly in the report and viva)

The system **never automatically disconnects, blocks, or reconfigures a life-critical device** (`device_types.is_life_critical = TRUE`). For those, it only alerts and recommends. Auto-mitigation (rate-limit, block) is permitted only for non-critical devices and only with the configured approval flag. Cutting network to a ventilator to "stop an attack" could kill a patient; the IDS must never be able to cause the harm it exists to prevent.

15. Testbed & attack generation

Attack	Tool / method	Command sketch
ICMP/UDP/SYN flood (DoS/DDoS)	hping3	<code>hping3 --flood -S -p 1883 <target></code>
Port scan / OS fingerprint (Recon)	Nmap	<code>nmap -sS -O <target></code>
ARP spoofing / MITM (Spoofing)	Ettercap / Bettercap	<code>bettercap -eval "set arp.spoof.targets <ip>; arp.spoof on"</code>
MQTT CONNECT flood	MQTTSA / custom paho loop	rapid connect/disconnect storm to broker
Data injection / alteration	custom MITM script	intercept vitals, rewrite HR/SpO ₂ before forward
Benign baseline	device simulators	normal publish cadence

Run all of this **only on your own isolated lab network**. Never point these tools at any network or device you do not own — that is both against the law and against the ethics you are claiming to uphold in this very project. State the isolated-testbed constraint in your report's ethics section.

16. Deployment

`docker-compose.yml` brings up: `mysql`, `redis`, `mosquitto`, `backend` (FastAPI+Uvicorn), `worker` (detection consumer), `frontend` (Vite build served by nginx). Simulators run on the host or in extra containers.

```
docker compose up -d
# seed reference data (roles, attack types, device_types)
docker compose exec backend python -m backend.seed
# start simulators
python simulator/device_monitor.py &
# open http://localhost:5173
```

Model files ship in `models/` and are volume-mounted so you can retrain and hot-swap by calling `/models/{id}/activate`.

17. Testing strategy

Level	What	Tool
Unit	feature extraction correctness (known packets → known features), rule bounds, risk score math	pytest
Unit	each model loads and returns the right output shape	pytest
Integration	flow → Redis → worker → detection row → alert row	pytest + test DB
Integration	<code>/detect</code> end to end with a crafted vector	pytest-asyncio + httpx
Model	accuracy/F1 thresholds must not regress below saved baselines	evaluate.py in CI
Load	replay 100k flows, assert throughput + p95 latency	locust / custom

Security	authz: a <code>viewer</code> cannot hit <code>/models/retrain</code> ; JWT expiry honoured	pytest
E2E demo	launch each attack, assert an alert of the right family appears within N seconds	scripted

Target metrics to report: detection latency (p50/p95 ms/flow), end-to-end alert time (attack start → alert on screen), throughput (flows/sec sustained).

18. Evaluation metrics (what goes in the results chapter)

Detection quality: accuracy, precision, recall, F1, macro-F1, per-class recall, ROC-AUC, PR-AUC, confusion matrices (binary, 6-class, 19-class). **Operational:** false positive rate (critical for hospitals), mean time to detect, inference latency, sustained throughput. **Zero-day:** detection rate on the left-out attack family (§9.3). **Generalisation:** cross-dataset accuracy drop (§9.4). **Explainability:** SHAP global + per-class + representative force plots; agreement between SHAP and LIME on top features. **Ablation:** the §9.5 table.

State your **target numbers** up front so you have something to hit: binary F1 ≥ 0.98, 6-class macro-F1 ≥ 0.95, FPR ≤ 1%, p95 inference ≤ 20 ms/flow on CPU, live alert within 5 s of attack onset. Published work on these datasets reaches the high-90s F1, so these are realistic, not fantasy.

19. Project timeline (map to your two semesters)

Phase	Weeks	Deliverable	Owner (by skill)
1. Literature + dataset study	1–3	survey table, dataset downloaded & understood	Rohit (research/IoMT)
2. Preprocessing + EDA	3–5	clean splits, EDA notebook, feature set v1	Akash (preprocessing)
3. Baseline + core models (M1,M2)	5–8	binary + 6-class results	Dipak (model training)
4. Deep models (M3,M4,M6)	8–11	CNN-BiLSTM, autoencoder, vitals LSTM	Dipak + Akash
5. Backend + DB + API	9–13	schema, FastAPI, <code>/detect</code> working	(assign)
6. Capture + gateway + simulators	11–14	live flow → vector working	(assign)
7. Frontend dashboard	13–17	all pages, WebSocket live	Shruti (design/docs) + dev
8. Integration + SHAP + alerts	16–19	full pipeline live	whole team
9. Zero-day + cross-dataset + ablation experiments	18–21	results chapter	Dipak + Rohit
10. Testing, hardening, demo prep	21–23	test suite, demo script	whole team
11. Report + paper + PPT + video	22–24	final submission	Shruti lead + all

Team-to-module mapping (from your synopsis skills)

- **Rohit** (research, IoMT, cybersec basics) → literature, attack taxonomy, testbed & attack generation, ethics section
- **Akash** (Python, preprocessing, ML basics) → data pipeline, feature engineering, autoencoder
- **Dipak** (Python, ML, training/testing) → all supervised models, evaluation, SHAP
- **Shruti** (documentation, system design, MS Office, communication) → system design docs, dashboard UX, report, PPT, presentation

20. Risks & mitigations

Risk	Mitigation
CICIoMT2024 is large; laptops run out of RAM	Work on per-protocol subsets; use <code>dtype</code> downcasting; sample for prototyping, full run on Colab/Kaggle
19-class accuracy disappoints	Report 6-class as headline, 19-class as "hard case" with honest discussion — this is expected in the literature
No physical IoMT devices	Simulators + WUSTL-EHMS dataset fully substitute; physical ESP32 is a bonus, not a requirement
Live capture needs admin/root	Run sniffer with capabilities set, or demo via pcap replay if lab machine is locked down
Class imbalance skews results	Resample train-only, report macro-F1 + per-class recall, never accuracy alone
Scope creep (too many models)	M1, M2, M4, M6 are the must-haves; M3, M5, M7–M10 are enhancements — build in that order
Overfitting / leakage giving fake 100%	Follow §6.5 leakage rules strictly; a suspiciously perfect score is a red flag to examiners

21. What I added beyond your synopsis (and why)

1. **IoMT-native datasets (CICIoMT2024, WUSTL-EHMS-2020).** Your synopsis listed only generic IT/IoT datasets. Without a healthcare dataset the "Medical" in the title is unsupported. (*verified against the CIC and WUSTL dataset pages*)
2. **The five-stage pipeline instead of one classifier.** Turns a coursework-level "train a model" into a system with a defensible architecture.
3. **Stage 3 zero-day autoencoder.** Your objectives promise detecting *unknown* attacks; a supervised classifier alone cannot do that. This closes that gap.
4. **Stage 4 physiological plausibility.** The single most novel and defensible piece — it links cybersecurity to patient safety, which is your stated justification.
5. **SHAP explainability.** Your synopsis never mentions it, but healthcare IDS research now treats interpretability as mandatory, and it is what makes an alert actionable.
6. **The safety rule (never auto-disconnect a life-critical device).** Shows engineering maturity and directly answers an obvious viva question.
7. **Full operational layer** — alert lifecycle, risk scoring, dedup, RBAC, audit — so it reads as a real system, not a notebook.
8. **Honest evaluation protocol** — leakage rules, zero-day hold-out, cross-dataset test, ablation. This is what separates a distinction-grade project from a demo.

22. Corrected & relevant references

Important: the 20 references in your current synopsis are all about **edge/cloud computing and the compute continuum** — none of them is about IoMT security, intrusion detection, or the datasets you use. In a viva this mismatch is the easiest thing for an examiner to attack. Replace them with the domain-correct references below. (*Dataset facts below were verified against primary sources during preparation of this document.*)

Datasets - S. Dadkhah, E. C. P. Neto, R. Ferreira, R. C. Molokwu, S. Sadeghi, A. A. Ghorbani, "CICIoMT2024: Attack Vectors in Healthcare devices — A Multi-Protocol Dataset for Assessing IoMT Device Security," *Internet of Things*, 2024. (18 attacks, 40 devices, 5 classes: DDoS/DoS/Recon/MQTT/Spoofing; Wi-Fi/MQTT/Bluetooth; ~3.2M instances, 46 features; 2/6/19-class labels.) - A. A. Hady, A. Ghubaish, T. Salman, D. Unal, R. Jain, "Intrusion Detection System for Healthcare Systems Using Medical and Network Data: A Comparison Study," *IEEE Access*, 2020. (WUSTL-EHMS-2020: EHMS testbed, 16,318 records, 44 features = 35 network + 8 biometric + 1 label; MITM spoofing & data injection.) - E. C. P. Neto et al., "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, 2023. (34 classes → 8 categories, 46 features.)

IoMT / IoT intrusion detection (representative — search these on IEEE Xplore/Springer and cite the ones you actually read) - Kilincer, Ertam, Sengur, "Machine learning methods for cyber security intrusion detection," survey. - Alani et al., explainable IoMT IDS using XGBoost + SHAP/LIME on WUSTL-EHMS-2020. - Thamilarasu, Odesile, Hoang, "An intrusion detection system for the Internet of Medical Things." - Ghubaish et al., "Recent advances in the Internet of Medical Things (IoMT) systems security." - Ravi, Chaganti, Alazab, deep-learning IDS for healthcare IoT.

Methods & explainability - Lundberg & Lee, "A unified approach to interpreting model predictions" (SHAP), NeurIPS 2017. - Ribeiro, Singh, Guestrin, "Why should I trust you?: Explaining the predictions of any classifier" (LIME), KDD 2016. - Chen & Guestrin, "XGBoost: A scalable tree boosting system," KDD 2016.

Keep your synopsis's reference format, but swap the content. Cite only papers your team has actually read, and add 8–12 more from IEEE Xplore, Springer, ScienceDirect and Google Scholar during your literature phase. Aim for 20–25 domain-correct references in the final report.

23. Viva / defence preparation (anticipate these)

Likely question	Your answer (from this doc)
Why not just a firewall or Snort?	Signatures miss zero-days and can't read intent inside legitimate MQTT; §1.2
What if the attack is brand new?	Stage 3 autoencoder trained on benign only; §8.5, §9.3
Where's the "Medical" part?	Stage 4 physiological plausibility + WUSTL biometric dataset; §8.7
Can it run in real time?	Report your measured p95 latency; tree models are sub-20 ms/flow on CPU; §18
How do you handle class imbalance?	Train-only resampling + class weights, report macro-F1; §6.4
Isn't 99% accuracy suspicious?	Discuss leakage controls and why you also report per-class recall and cross-dataset drop; §6.5, §9.4
Can it act on attacks, not just detect?	Advisory mitigation + strict safety rule never to auto-disconnect life-critical devices; §14.3
Why should a doctor trust it?	SHAP explanation on every alert; §8.10
What's your actual novelty?	The hybrid (supervised + unsupervised + physiological) + explainability + IoMT-specific engineered features; §21

24. Build order (do it in this sequence)

- 1. Repo skeleton + Docker Compose (MySQL, Redis, Mosquitto) + schema migrations
- 2. Download CICIoMT2024 + WUSTL-EHMS-2020; EDA notebook
- 3. Preprocessing pipeline → saved splits, scaler, feature order
- 4. M1 (RF) + M2 (XGBoost) → binary + 6-class results ← *you now have a real result to show*
- 5. FastAPI backend + `/detect` loading M1/M2 → detections landing in DB
- 6. Simulators + gateway + sniffer + feature extractor → live flow → `/detect`
- 7. M4 autoencoder (Stage 3) + M6 vitals LSTM (Stage 4)
- 8. Alert service (dedup, risk score, lifecycle) + WebSocket
- 9. React dashboard (Overview → Alerts → Devices → Models)
- 10. SHAP layer + Models page
- 11. M3 CNN-BiLSTM + M10 stacking (enhancements)
- 12. Zero-day, cross-dataset, ablation experiments → results chapter
- 13. Test suite, demo script, hardening
- 14. Report + paper + PPT + presentation video

Get to step 4 fast — a working binary classifier with real numbers is the foundation everything else hangs on, and it de-risks the whole project early.

End of build reference document. Everything needed to construct the project — rationale, architecture, tech stack, datasets, all models with per-model functionality, the complete database schema with DDL, module functionality, APIs, dashboard, detection logic, testing, evaluation, timeline, corrected references, and viva prep — is contained above.